

Estructuras de programación

Objetivos

- Comprender la diferencia entre análisis léxico, sintaxis y semántica.
- Identificar los componentes léxicos de Java.
- Construir y evaluar expresiones matemáticas y lógicas.
- Utilizar métodos de la biblioteca estándar de Java.
- Reconocer los distintos tipos de sentencias.
- Entender el concepto de flujo de control.
- Desarrollar algoritmos iterativos.
- Realizar trazas de ejecución.

Contenido

Introducción	3
Análisis léxico y componentes del lenguaje	3
Concepto de Token y separador	4
Palabras reservadas	8
Expresiones y Operadores	11
Operadores	11
Evaluación en cortocircuito	11
Precedencia y asociatividad	11
Conversión de tipos	11
Reglas de promoción	11
Conversión explícita	11
Clases envolventes	11
Uso de la biblioteca estándar	11
La clase Math	11
Instrucciones	11
Sentencias	11
Bloques y ámbito	11
El flujo de ejecución	11
Concepto de flujo de control	11
Estructuras secuenciales	11
Selección	11
Selección simple	11
Selección compuesta	11
Selección múltiple	11
Lógica de anidamiento	11
Repetición	11
Sentencia while	12
Bucles controlados por condición	12
Bucles controlados por contador	12
Diseño de bucles	12
Depuración y verificación	12
Trazas de ejecución	12
Errores comunes	12

Introducción

Hasta este punto de nuestro aprendizaje, hemos comprendido que un programa de ordenador es, en esencia, un algoritmo escrito para ser ejecutado por una máquina. En el tema anterior, exploramos los cimientos: aprendimos a declarar variables, a reconocer tipos de datos primitivos y a utilizar la biblioteca estándar de Java para realizar operaciones de entrada/salida básicas. Sin embargo, si analizamos los programas que hemos construido, observaremos que todos comparten una característica común: son puramente lineales. El ordenador se limita a ejecutar una instrucción tras otra, en el estricto orden físico en que fueron escritas.

No obstante, la programación en el «mundo real» y la resolución de problemas técnicos complejos en entornos productivos requieren una sofisticación mucho mayor. El pensamiento computacional no es una simple sucesión de pasos, sino un tejido o «urdimbre» donde cada hebra lógica debe entrelazarse con precisión. Para lograr esto, un lenguaje de programación debe ser entendido como un conjunto de reglas, símbolos y palabras que nos permiten modelar la realidad.

El proceso por el cual el ordenador interpreta nuestras intenciones comienza con el análisis léxico. Cuando el compilador recibe nuestro código fuente, su primera tarea es identificar tokens, elementos con significado propio que el compilador separa gracias a delimitadores. Como sucede en el lenguaje natural, no podemos formar una oración coherente amontonando palabras, en los lenguajes de programación combinamos estos componentes léxicos para formar expresiones. Algo como $n + 1$, por sí solo, es solo un valor latente; para que sea algo capaz de realizar una tarea, debe integrarse en una estructura que le dé sentido, como una asignación, por ejemplo.

La verdadera potencia del software reside en la capacidad de romper la linealidad del código para establecer un flujo de control. Este define el orden en que las sentencias se ejecutan realmente durante la actividad de la aplicación. Para ilustrarlo, podemos usar la analogía de la conducción: ir por un tramo recto de carretera equivale a una estructura secuencial; sin embargo, al llegar a una bifurcación, debemos tomar una decisión basada en una condición o quizás necesitamos dar varias vueltas hasta encontrar un lugar donde aparcar.

Profundizaremos aquí en cómo construir expresiones y conoceremos la selección y la iteración, herramientas que nos permitirán empezar a desarrollar algoritmos inteligentes, capaces de tomar decisiones autónomas y procesar información de forma eficiente, sentando así las bases necesarias antes de adentrarnos en los fundamentos de la programación orientada a objetos.

Análisis léxico y componentes del lenguaje

Como hemos señalado, la construcción de un programa comienza con la escritura de **código fuente**, una tarea que realizamos utilizando un editor y siguiendo las reglas gramaticales del lenguaje. Sin embargo, para que este texto cobre vida y se convierta en

una serie de acciones ejecutables, debe pasar por un proceso de transformación riguroso. El primer paso de este viaje técnico es el **análisis léxico**, una fase en la que el compilador actúa como un lector meticuloso que descompone la secuencia ininterrumpida de caracteres que le entregamos en piezas individuales dotadas de significado.

En el lenguaje natural, cuando leemos una frase, nuestro cerebro identifica automáticamente dónde termina una palabra y comienza la siguiente, reconociendo sustantivos, verbos y adjetivos. En el ámbito de la programación, el ordenador realiza una operación análoga. No percibe el código como un conjunto de instrucciones complejas de un solo vistazo, sino como un flujo de símbolos pertenecientes al alfabeto *Unicode*, en el caso de Java. La labor del analizador léxico consiste en agrupar estos símbolos en unidades atómicas, que son los componentes básicos que conforman la estructura del lenguaje.

Esta etapa es fundamental porque establece los cimientos sobre los que se construirá la lógica del programa, permitiendo ensamblar estos bloques en estructuras sintácticamente correctas. Si el analizador léxico encuentra un símbolo inesperado o una combinación de caracteres que no encaja en las categorías permitidas, el proceso se detendrá antes incluso de intentar comprender la intención del algoritmo. Por lo tanto, antes de profundizar en cómo tomar decisiones o repetir procesos, es esencial comprender la naturaleza de estos elementos mínimos y los mecanismos que permiten al compilador distinguirlos unos de otros.

Analizador léxico: Fase del compilador que descompone el flujo continuo de caracteres del código fuente en unidades atómicas con significado (*tokens*).

Concepto de Token y separador

Dentro del proceso de análisis léxico, el compilador debe ser capaz de reconocer los componentes básicos que integran el código fuente para poder interpretarlos adecuadamente. Estos componentes mínimos, que no pueden ser descompuestos en partes más pequeñas sin perder su sentido, reciben el nombre de **tokens** o elementos atómicos. En la gramática de un lenguaje de programación como Java, los *tokens* son el equivalente a las palabras en una oración del lenguaje natural; cada uno de ellos cumple una función específica, ya sea representar un valor constante, nombrar una variable o indicar una operación matemática.

Para que el analizador léxico pueda distinguir estos elementos dentro de la secuencia ininterrumpida de caracteres que escribimos, el lenguaje emplea unos símbolos especiales denominados **separadores**. Un separador es cualquier carácter o marca que indica de manera inequívoca dónde termina un *token* con significado propio y dónde comienza el siguiente. En este sentido, la programación vuelve a imitar a la lengua: así como usamos espacios para separar palabras en un texto escrito para que no se conviertan en una masa de letras incomprensible, Java utiliza los separadores para evitar que los *tokens* se fusionen.

Token: Unidad mínima o componente atómico que posee significado propio dentro de la gramática del lenguaje de programación.

Separador: Carácter o marca que delimita inequívocamente el fin de un *token* y el inicio del siguiente.

Los elementos que actúan como separadores en Java son los siguientes:

- **Espacio en blanco:** es el separador más común y sencillo, utilizado de la misma forma que en la escritura convencional.
- **Tabulador:** actúa de forma equivalente al espacio en blanco desde el punto de vista del compilador.
- **Retorno de carro o salto de línea:** indica el fin de una línea física, pero léxicamente tiene el mismo valor que un espacio. El compilador es capaz de tratar un programa completo escrito en una sola línea o con cada palabra en una línea distinta con el mismo resultado técnico, ya que sustituye cualquier secuencia de estos separadores por un único espacio en blanco¹.

Una vez que el analizador léxico ha hecho uso de estos separadores para aislar los *tokens*, estos se clasifican en diferentes categorías según su naturaleza, como los literales, los identificadores, los operadores y las palabras reservadas. Solo si el ensamblaje de estos bloques atómicos sigue las reglas de la sintaxis, el compilador podrá avanzar hacia la fase de traducción y ejecución del programa. ### Literales

Un **literal** —o valor literal— es la representación explícita y constante de un valor concreto directamente escrito en el código fuente de un programa. A través de los literales, el programador especifica valores fijos para los tipos de datos primitivos, el tipo de datos `String` y la referencia especial `null`.

Desde el punto de vista del análisis léxico, los literales son identificados por el compilador como *tokens* atómicos. Esto significa que constituyen unidades de información indivisibles con significado propio dentro de la gramática del lenguaje. A diferencia de lo que ocurre con un identificador o una variable, cuyo contenido o estado puede fluctuar dinámicamente durante el ciclo de vida de la ejecución, el valor representado por un literal queda prefijado de manera inmutable desde la fase de compilación.

Literal: *Token* que representa la expresión gráfica e inalterable de un valor constante de un tipo de datos primitivo, de un objeto `String` o de la referencia nula `null` dentro del código fuente.

Dentro de la especificación sintáctica de Java, los literales se clasifican rigurosamente según la categoría de datos a la que corresponden, obedeciendo a un conjunto estricto de convenciones léxicas para su escritura:

1. Como vimos en el capítulo anterior, esto no es así cuando se trata de una cadena, por ejemplo.

Literales enteros

De forma predeterminada, cuando el analizador léxico procesa un literal numérico compuesto únicamente por dígitos (sin coma ni punto fraccionario), el compilador deduce que se trata de un valor de tipo `int` codificado en 32 bits. Para aportar versatilidad en entornos de programación de bajo nivel, diseño de sistemas o manipulación de máscaras de bits, Java permite representar literales enteros mediante cuatro sistemas de numeración:

- **Decimal (base 10):** es la representación convencional mediante una secuencia de dígitos del 0 al 9. Por regla léxica, un literal decimal no debe comenzar con el dígito 0 (salvo el propio número cero), para evitar ambigüedades con la notación octal.
- **Octal (base 8):** se especifica anteponiendo un cero (0) como prefijo al número, seguido exclusivamente por dígitos comprendidos entre el 0 y el 7 (por ejemplo, 012, que equivale al valor decimal 10).
- **Hexadecimal (base 16):** se denota mediante el prefijo 0x o 0X, aceptando dígitos del 0 al 9 y letras de la a a la f (tanto en minúsculas como en mayúsculas). Un ejemplo es 0x1A, correspondiente al decimal 26.
- **Binario (base 2):** permite la notación explícita mediante el prefijo 0b o 0B, empleando únicamente los dígitos 0 y 1 (por ejemplo, 0b1010, equivalente al valor decimal 10)².

Si se requiere que el literal sea interpretado por el compilador como un entero de precisión extendida de 64 bits (`long`), es obligatorio añadir el sufijo `L` o `l` al final de la secuencia de dígitos. Se recomienda enfáticamente el uso de la letra `L` mayúscula para evitar confusiones visuales con el número uno (1).

```
int contador = 100;           // Decimal
int mascaraHex = 0xFF;        // Hexadecimal (255 decimal)
int patronBits = 0b1100;      // Binario (12 decimal)
long poblacion = 8000000000L; // Literal tratado como 'long'
```

Internamente, la máquina virtual Java (JVM) almacena todos los valores enteros utilizando la representación binaria en **complemento a dos con signo**³. Este formato fija los límites inferior y superior de los rangos numéricos que cada tipo de entero puede albergar en memoria (por ejemplo, entre -2^{31} y $2^{31} - 1$ para el tipo `int`).

Literales de punto flotante

Los literales de punto flotante se emplean para expresar números reales o fraccionarios. Por defecto, la gramática de Java interpreta cualquier literal con punto decimal o

2. A partir de Java 9 (JEP 213), el carácter solitario `_` dejó de ser un identificador válido y pasó a ser un término reservado.

3. El sistema de representación en complemento a dos asigna el bit más significativo (MSB) como bit de signo, permitiendo operar aritméticamente con números enteros positivos y negativos de manera unificada a nivel de procesador.

notación exponencial como un valor de tipo `double` (64 bits), garantizando un grado elevado de precisión matemática.

Existen dos formas estándar de escribir literales de punto flotante:

- **Notación decimal:** utiliza el carácter punto (.) para separar explícitamente la parte entera de la fraccionaria (por ejemplo, 3.14159 o .5).
- **Notación científica o exponencial:** añade la letra e o E seguida de un exponente entero (positivo o negativo) que representa una potencia de diez (por ejemplo, 1.5e3 para representar 1.5×10^3 , es decir, 1500.0, o 2.5E-4 para 2.5×10^{-4}).

Cuando sea preciso declarar un literal de precisión simple de 32 bits (`float`) —para optimizar la memoria o cumplir con los requerimientos de un método—, se debe añadir obligatoriamente el sufijo **f** o **F** al final del número.

```
double pi = 3.1415926535;    // Tipo 'double' por defecto
double masa = 5.972e24;      // Notación científica (double)
float gravedad = 9.81f;      // Sufijo 'f' para forzar el tipo 'float'
```

La codificación interna de estos datos sigue escrupulosamente la norma **IEEE 754** (formato binario de punto flotante para ordenadores), estructurando el valor en tres campos binarios diferenciados: bit de signo, mantisa y exponente. Esto hace posible representar valores extremadamente amplios o infinitesimales, asumiendo las limitaciones de redondeo y precisión propias de la aritmética de punto flotante.

Literales booleanos, de carácter, de cadena y nulo

Para la representación explícita de valores no numéricos o de estructuras complejas, el lenguaje provee la siguiente gama de literales:

- **Literales booleanos:** representan los valores de la lógica booleana y están constituidos únicamente por las dos palabras reservadas `true` (verdadero) y `false` (falso). A diferencia de otros lenguajes como C, en Java no existe una equivalencia numérica directa entre el valor 0 o 1 y los valores booleanos.
- **Literales de carácter:** representan un único símbolo del juego de caracteres Unicode. Se delimitan mediante comillas simples ('a', '8', 'Z'). Incluyen además el soporte para **secuencias de escape** (combinaciones que comienzan con la barra invertida \) para representar caracteres de control o no imprimibles ('\n' para salto de línea, '\t' para tabulación, '\\' para comilla simple, '\\\' para barra invertida) o directamente notación hexadecimal Unicode ('\u0041' para la letra 'A').
- **Literales de cadena (String):** consisten en una secuencia de cero o más caracteres delimitados por comillas dobles ("Hola mundo"). Aunque `String` no es un tipo de datos primitivo sino una clase, Java proporciona este soporte sintáctico especial, convirtiendo automáticamente cualquier literal de este tipo en una instancia inmutable de la clase `java.lang.String`.

- **Literal nulo (null):** formado por la palabra reservada `null`, representa la ausencia explícita de referencia a un objeto dentro de una variable de tipo objeto o referencia.

```
boolean activo = true;
char inicial = 'J';
char salto = '\n';
char unicodeA = '\u0041';
String saludo = "Bienvenido a Java";
Object objetoVacio = null;
```

Uso del carácter subrayado (_) en literales numéricos: A partir de Java 7, se permite intercalar el carácter subrayado (_) entre los dígitos de cualquier literal numérico para mejorar la legibilidad del código fuente (por ejemplo, `1_000_000` o `0b1101_0010`). Estos caracteres son ignorados por el analizador léxico durante la fase de compilación y no afectan al valor asignado.

Palabras reservadas

Como ya comentamos en el capítulo anterior, son aquellos *tokens* que tienen asignada una función específica dentro del lenguaje: los programadores tienen prohibido utilizarlas para cualquier otro propósito; por ello, no es posible nombrar una variable, un campo o una clase utilizando uno de estos términos. Al igual que en el lenguaje natural existen palabras con funciones gramaticales fijas, como las preposiciones, las palabras reservadas constituyen el vocabulario fundamental que el compilador reconoce para estructurar la lógica del programa.

Gran parte del proceso de aprendizaje de un lenguaje de programación consiste en familiarizarse con el significado y la utilidad de estos términos. En el caso de Java, el conjunto de estas palabras ha ido evolucionando con las diferentes versiones del lenguaje para adaptarse a nuevas necesidades técnicas. En la actualidad, el núcleo está compuesto por cincuenta y una palabras, a las que se suman dieciséis términos adicionales cuyo significado especial solo se activa dependiendo del contexto específico en el que se utilicen.

Es importante destacar que Java es un lenguaje extremadamente estricto con la forma en que se escriben estos elementos, siendo sensible a la diferencia entre mayúsculas y minúsculas. Todas las palabras reservadas se componen exclusivamente de letras minúsculas. Por esta razón, un término como **int** es reconocido como una palabra reservada para definir tipos enteros, mientras que `Int` o `INT` serían tratados como identificadores diferentes definidos por el usuario, aunque su uso se desaconseja para evitar ambigüedades en la lectura del código.

Dentro de este grupo, existen casos particulares como las palabras **goto** y **const**. Aunque figuran en la lista oficial y el programador no puede emplearlas como nombres de variables, actualmente no tienen ninguna función operativa dentro del lenguaje Java. Su reserva responde principalmente a razones históricas y al deseo de los diseñadores

del lenguaje de evitar que programadores provenientes de otros entornos, como C o C++, intenten aplicar estructuras de programación que son incompatibles con la seguridad y la filosofía de Java.

A continuación, se presenta la relación de palabras reservadas en Java, estructurada conforme a la [Especificación del Lenguaje Java \(JLS\)](https://docs.oracle.com/javase/7/docs/technotes/guides/language/keywords.html)⁴:

Palabras reservadas (*Keywords*) en la Especificación del Lenguaje Java (JLS) ⁴					
abstract	assert	boolean	break	byte	case
catch	char	class	const	continue	default
do	double	else	enum	extends	final
finally	float	for	goto	if	implements
import	instanceof	int	interface	long	native
new	package	private	protected	public	return
short	static	strictfp ⁵	super	switch	synchronized
this	throw	throws	transient	try	void
volatile	while	—			

Asimismo, existen los siguientes términos que funcionan como palabras reservadas dependiendo del contexto en el que se utilicen:

Palabras reservadas dependientes del contexto (*Contextual Keywords*)			
exports	module	non-sealed	open
opens	permits	provides	record
requires	sealed	to	transitive
uses	var	with	yield

4. La documentación y especificación oficial actualizada de Java puede consultarse en la plataforma [Oracle Java Specification](https://docs.oracle.com/javase/7/docs/technotes/guides/language/keywords.html).

5. La palabra strictfp fue introducida para restringir los cálculos de punto flotante a la norma IEEE 754; su uso es obsoleto.

Finalmente, conviene recordar que el carácter de subrayado (_) por sí solo también se considera un elemento reservado para usos futuros⁶, lo que prohíbe su empleo como un identificador de una sola letra en las declaraciones del programa.

6. A partir de Java 9 (JEP 213), el carácter solitario `_` dejó de ser un identificador válido y pasó a ser un término reservado.

Expresiones y Operadores

Operadores

Evaluación en cortocircuito

Precedencia y asociatividad

Conversión de tipos

Reglas de promoción

Conversión explícita

Clases envolventes

Uso de la biblioteca estándar

La clase Math

Instrucciones

Sentencias

Bloques y ámbito

El flujo de ejecución

Concepto de flujo de control

Estructuras secuenciales

Selección

Selección simple

Selección compuesta

Selección múltiple

Lógica de anidamiento

Repetición

Sentencia while

Bucles controlados por condición

Bucles controlados por contador

Diseño de bucles

Depuración y verificación

Trazas de ejecución

Errores comunes